

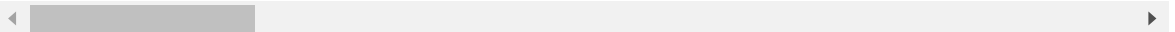
```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import GridSearchCV, ShuffleSplit
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, ShuffleSplit
```

```
In [2]: df = pd.read_csv("C:/Users/THUSHAN/df_cleaned.csv")
df.head()
```

Out[2]:

	account_open_date	age	location	occupation	income_level	fico_score	debt_to_income_ratio
0	2020-07-12	46	Miami	Engineer	71936	566.0	1.508
1	2024-07-27	60	Houston	Lawyer	8574	787.0	1.182
2	2022-03-06	25	Chicago	Teacher	59349	688.0	1.121
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	1.659
4	2022-09-10	36	Los Angeles	Other	6197	758.0	1.008

5 rows × 24 columns



```
In [3]: df.shape
```

Out[3]: (6720, 24)

In [4]: df.dtypes

```
Out[4]: account_open_date      object
age                int64
location           object
occupation         object
income_level       int64
fico_score         float64
debt_to_income_ratio float64
payment_methods_high_risk bool
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
earliest_credit_account object
recent_trade_activity object
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag bool
public_records_flag bool
date_contradiction_flag bool
delinquency_status_log float64
number_of_credit_applications_log float64
max_balance_log float64
avg_balance_last_12months_capped float64
charge_off_status bool
dtype: object
```

```
In [5]: date_columns = ['account_open_date', 'earliest_credit_account', 'recent_trade_activity']
df[date_columns] = df[date_columns].apply(pd.to_datetime, errors='coerce')
```

In [6]: df.dtypes

```
Out[6]: account_open_date      datetime64[ns]
age                int64
location           object
occupation         object
income_level       int64
fico_score         float64
debt_to_income_ratio float64
payment_methods_high_risk bool
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
earliest_credit_account datetime64[ns]
recent_trade_activity datetime64[ns]
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag bool
public_records_flag bool
date_contradiction_flag bool
delinquency_status_log float64
number_of_credit_applications_log float64
max_balance_log float64
avg_balance_last_12months_capped float64
charge_off_status bool
dtype: object
```

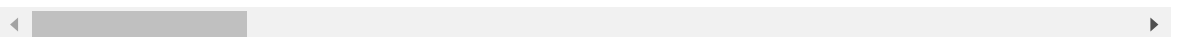
```
In [7]: df['recent_trade_activity_year'] = df['recent_trade_activity'].dt.year
df['recent_trade_activity_month'] = df['recent_trade_activity'].dt.month
df['recent_trade_activity_day_of_week'] = df['recent_trade_activity'].dt.weekday
df['earliest_credit_account_year'] = df['earliest_credit_account'].dt.year
df['earliest_credit_account_month'] = df['earliest_credit_account'].dt.month
df['earliest_credit_account_day_of_week'] = df['earliest_credit_account'].dt.weekday
df['account_open_date_year'] = df['account_open_date'].dt.year
df['account_open_date_month'] = df['account_open_date'].dt.month
df['account_open_date_day_of_week'] = df['account_open_date'].dt.weekday
```

```
In [8]: df
```

```
Out[8]:
```

	account_open_date	age	location	occupation	income_level	fico_score	debt_to_income
0	2020-07-12	46	Miami	Engineer	71936	566.0	1
1	2024-07-27	60	Houston	Lawyer	8574	787.0	1
2	2022-03-06	25	Chicago	Teacher	59349	688.0	1
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	1
4	2022-09-10	36	Los Angeles	Other	6197	758.0	1
...	...	...	...	...	...	...	...
6715	2021-01-17	65	San Francisco	Artist	10876	447.0	2
6716	2024-09-15	29	Boston	Consultant	16347	469.0	1
6717	2019-12-27	74	Chicago	Doctor	51012	580.0	2
6718	2021-04-10	33	Boston	Consultant	14566	589.0	1
6719	2024-06-20	68	Houston	Other	4501	321.0	1

6720 rows × 33 columns



```
In [9]: df = df.drop(['recent_trade_activity', 'earliest_credit_account', 'account_open_date'])
```

In [10]: `df.dtypes`

```
Out[10]: age                int64
location            object
occupation          object
income_level        int64
fico_score          float64
debt_to_income_ratio float64
payment_methods_high_risk bool
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag bool
public_records_flag bool
date_contradiction_flag bool
delinquency_status_log float64
number_of_credit_applications_log float64
max_balance_log float64
avg_balance_last_12months_capped float64
charge_off_status bool
recent_trade_activity_year int64
recent_trade_activity_month int64
recent_trade_activity_day_of_week int64
earliest_credit_account_year int64
earliest_credit_account_month int64
earliest_credit_account_day_of_week int64
account_open_date_year int64
account_open_date_month int64
account_open_date_day_of_week int64
dtype: object
```

In [11]: `dummy_df = pd.get_dummies(df[["location", "occupation"]])`
`dummy_df.head()`

```
Out[11]:
```

	location_Atlanta	location_Boston	location_Chicago	location_Dallas	location_Houston	location_Los_Angeles
0	0	0	0	0	0	0
1	0	0	0	0	0	1
2	0	0	1	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0

5 rows × 21 columns

In [12]: `# Convert multiple boolean columns to 0 and 1`
`boolean_columns = ['payment_methods_high_risk', 'multiple_applications_short_time_period']`
`df[boolean_columns] = df[boolean_columns].astype(int)`

```
In [13]: df.dtypes
```

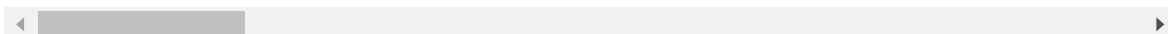
```
Out[13]: age                int64
location            object
occupation          object
income_level        int64
fico_score          float64
debt_to_income_ratio float64
payment_methods_high_risk int32
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
new_accounts_opened_last_12months int64
multiple_applications_short_time_period int32
unusual_submission_pattern int32
applications_submitted_during_odd_hours int32
watchlist_blacklist_flag int32
public_records_flag int32
date_contradiction_flag int32
delinquency_status_log float64
number_of_credit_applications_log float64
max_balance_log float64
avg_balance_last_12months_capped float64
charge_off_status int32
recent_trade_activity_year int64
recent_trade_activity_month int64
recent_trade_activity_day_of_week int64
earliest_credit_account_year int64
earliest_credit_account_month int64
earliest_credit_account_day_of_week int64
account_open_date_year int64
account_open_date_month int64
account_open_date_day_of_week int64
dtype: object
```

In [14]: df

Out[14]:

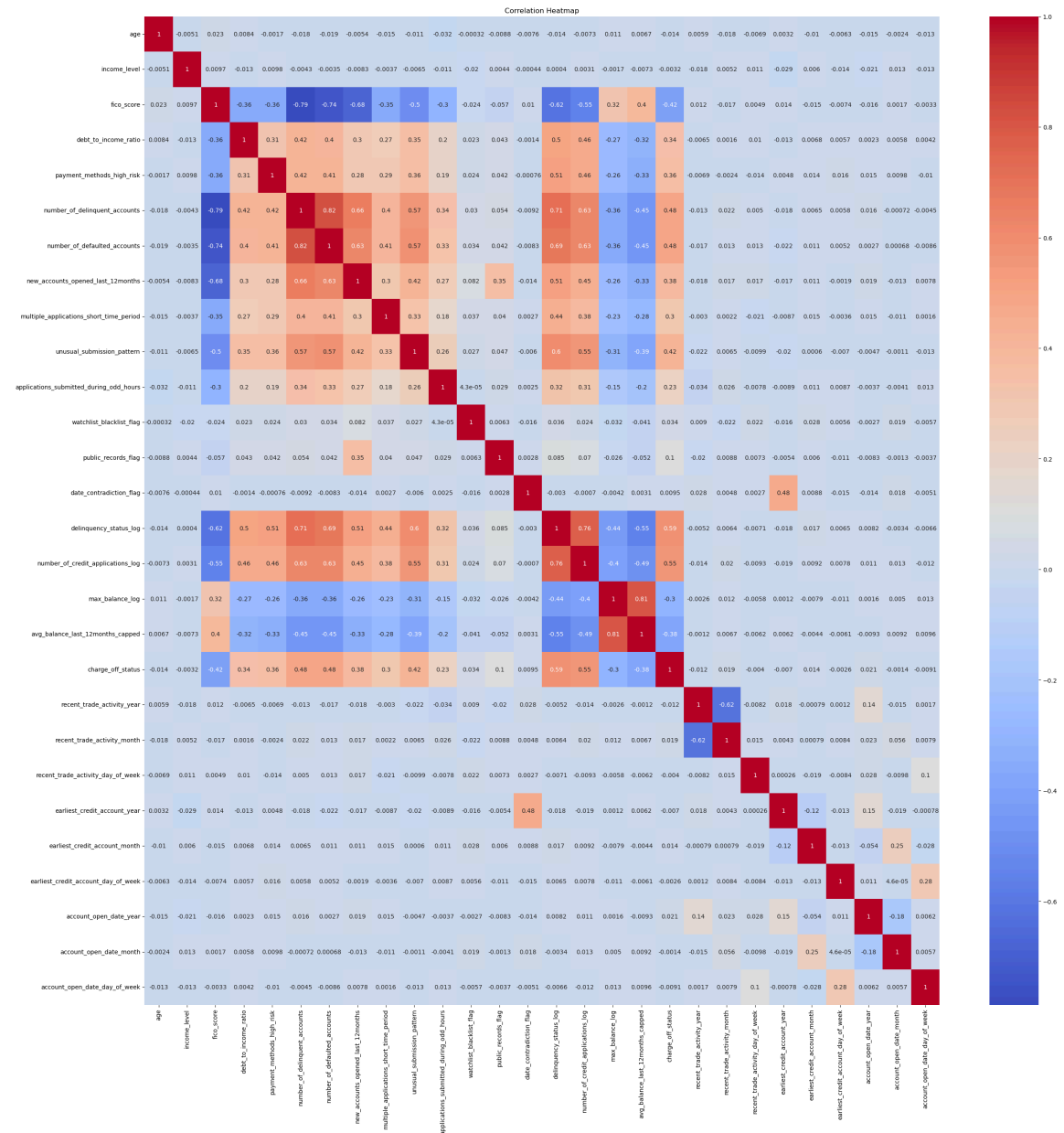
	age	location	occupation	income_level	fico_score	debt_to_income_ratio	payment_m
0	46	Miami	Engineer	71936	566.0	1.508626	
1	60	Houston	Lawyer	8574	787.0	1.182380	
2	25	Chicago	Teacher	59349	688.0	1.121196	
3	38	Los Angeles	Banker	6925	740.0	1.659631	
4	36	Los Angeles	Other	6197	758.0	1.008080	
...	...	...	...	...	...	...	
6715	65	San Francisco	Artist	10876	447.0	2.214227	
6716	29	Boston	Consultant	16347	469.0	1.516757	
6717	74	Chicago	Doctor	51012	580.0	2.648878	
6718	33	Boston	Consultant	14566	589.0	1.727134	
6719	68	Houston	Other	4501	321.0	1.750392	

6720 rows × 30 columns



In [110]:

```
# Drop non-numeric columns and calculate correlation
corr = df.drop(['location', 'occupation'], axis=1).corr()
plt.figure(figsize=(30, 30))
# Plot the heatmap
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



In [15]: df_train = pd.concat([df.drop(['location', 'occupation'], axis=1), dummy_df]

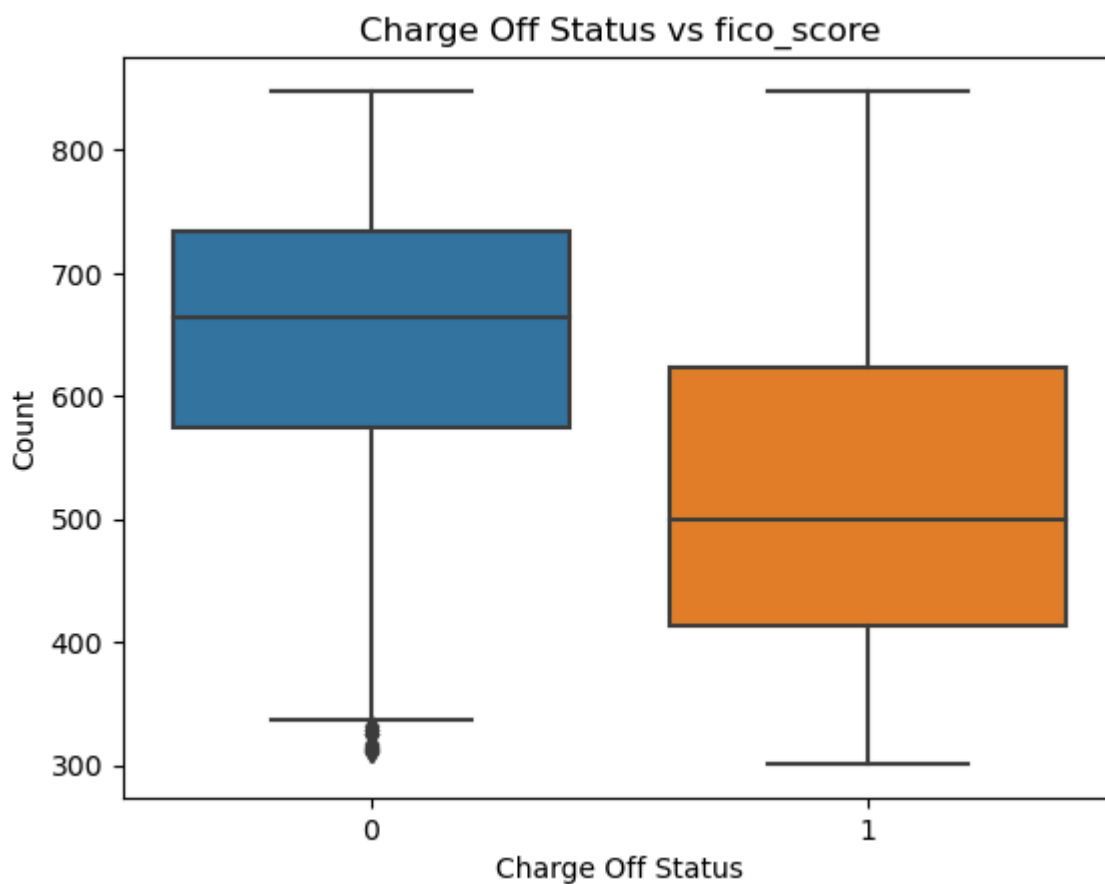
```
In [16]: df_train
```

```
Out[16]:
```

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	numt
0	46	71936	566.0	1.508626	0	
1	60	8574	787.0	1.182380	0	
2	25	59349	688.0	1.121196	1	
3	38	6925	740.0	1.659631	0	
4	36	6197	758.0	1.008080	0	
...	...	...	...	...	...	...
6715	65	10876	447.0	2.214227	1	
6716	29	16347	469.0	1.516757	1	
6717	74	51012	580.0	2.648878	1	
6718	33	14566	589.0	1.727134	1	
6719	68	4501	321.0	1.750392	1	

6720 rows × 49 columns

```
In [17]: sns.boxplot(x=df_train['charge_off_status'], y=df_train['fico_score'])  
plt.title('Charge Off Status vs fico_score')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```



In [179]:

```
df_train .to_csv('df_train.csv', index=False)
```

In [18]:

```
# Calculate correlations with the target variable  
correlation_with_charge_off_status = df_train.corr()['charge_off_status'].di  
correlation_with_charge_off_status=correlation_with_charge_off_status.to_dic  
correlation_with_charge_off_status
```

Out[18]:

```
{'age': -0.01379964596439038,  
'income_level': -0.0031971361556222396,  
'fico_score': -0.4250838156505667,  
'debt_to_income_ratio': 0.34031445574543234,  
'payment_methods_high_risk': 0.36015294359594024,  
'number_of_delinquent_accounts': 0.48144867571323413,  
'number_of_defaulted_accounts': 0.47835838804201797,  
'new_accounts_opened_last_12months': 0.3758655651010595,  
'multiple_applications_short_time_period': 0.3028251295919558,  
'unusual_submission_pattern': 0.33853378106191384,  
'applications_submitted_during_odd_hours': 0.2273979408043342,  
'watchlist_blacklist_flag': 0.034469923030702516,  
'public_records_flag': 0.10417007164680533,  
'date_contradiction_flag': 0.009515752637041096,  
'delinquency_status_log': 0.585363564238141,  
'number_of_credit_applications_log': 0.5473437934831684,  
'max_balance_log': -0.30085705928158485,  
'avg_balance_last_12months_capped': -0.3826428913288685,  
'recent_trade_activity_year': -0.012210972781350754,  
'recent_trade_activity_month': 0.018997266919417,  
'recent_trade_activity_day_of_week': -0.004039075120084077,  
'earliest_credit_account_year': -0.006992340947340053,  
'earliest_credit_account_month': 0.01398864360491933,  
'earliest_credit_account_day_of_week': -0.0026254455267813387,  
'account_open_date_year': 0.020848750294348507,  
'account_open_date_month': -0.0014124609071268475,  
'account_open_date_day_of_week': -0.009122250378774401,  
'location_Atlanta': 0.0019357618946704977,  
'location_Boston': -0.0095781283127651,  
'location_Chicago': -0.0026186170030702404,  
'location_Dallas': -0.012051389756155212,  
'location_Houston': -0.01997009982827969,  
'location_Los Angeles': -0.0028669779018248177,  
'location_Miami': 0.003390703203628185,  
'location_New York': 0.019802292640168896,  
'location_San Francisco': 0.0033337677380252248,  
'location_Seattle': 0.018902878653620166,  
'occupation_Accountant': 0.010882448524006468,  
'occupation_Artist': 0.0007448995623227031,  
'occupation_Banker': 0.0003005864122850876,  
'occupation_Consultant': 0.01046027513622544,  
'occupation_Doctor': 0.01196344030738049,  
'occupation_Engineer': 0.0008180255841824041,  
'occupation_Lawyer': -0.018301075294872782,  
'occupation_Other': 2.2453623067193566e-05,  
'occupation_Retail Worker': -0.01591469771206604,  
'occupation_Software Developer': -0.004469735028312866,  
'occupation_Teacher': -0.006194028387015281}
```

```
In [19]: total_corr = sum(abs(corr) for corr in correlation_with_charge_off_status.values())
total_corr
```

```
Out[19]: 5.576100022056865
```

```
In [20]: allocated_scores = {var: abs(corr) / total_corr * 1000 for var, corr in correlation_with_charge_off_status.items()}
```

```
In [21]: allocated_scores
```

```
Out[21]: {'age': 2.4747845106444273,
'income_level': 0.5733642049058701,
'fico_score': 76.23317622874443,
'debt_to_income_ratio': 61.03090948858195,
'payment_methods_high_risk': 64.58868064979401,
'number_of_delinquent_accounts': 86.34147052757518,
'number_of_defaulted_accounts': 85.78726818920389,
'new_accounts_opened_last_12months': 67.40653209488401,
'multiple_applications_short_time_period': 54.30769326125757,
'unusual_submission_pattern': 60.711568968061364,
'applications_submitted_during_odd_hours': 40.78082170420852,
'watchlist_blacklist_flag': 6.181726097873607,
'public_records_flag': 18.681528529751866,
'date_contradiction_flag': 1.7065247394057692,
'delinquency_status_log': 104.97723532983129,
'number_of_credit_applications_log': 98.15889085886032,
'max_balance_log': 53.954745806479856,
'avg_balance_last_12months_capped': 68.62195617282387,
'recent_trade_activity_year': 2.1898769270724943,
'recent_trade_activity_month': 3.4069092814460396,
'recent_trade_activity_day_of_week': 0.72435485448738,
'earliest_credit_account_year': 1.2539841322216412,
'earliest_credit_account_month': 2.5086787449266943,
'earliest_credit_account_day_of_week': 0.4708390302175545,
'account_open_date_year': 3.7389484069293997,
'account_open_date_month': 0.2533062358170955,
'account_open_date_day_of_week': 1.6359552989886041,
'location_Atlanta': 0.3471533665130437,
'location_Boston': 1.7177109942213697,
'location_Chicago': 0.46961442454619146,
'location_Dallas': 2.1612578161232117,
'location_Houston': 3.5813740336948414,
'location_Los Angeles': 0.5141546762942159,
'location_Miami': 0.6080779021566853,
'location_New York': 3.5512800275889584,
'location_San Francisco': 0.5978672772794869,
'location_Seattle': 3.3899819907906585,
'occupation_Accountant': 1.9516236224170602,
'occupation_Artist': 0.13358791258696448,
'occupation_Banker': 0.05390620883701613,
'occupation_Consultant': 1.8759123930432908,
'occupation_Doctor': 2.1454852423840696,
'occupation_Engineer': 0.14670210020383703,
'occupation_Lawyer': 3.282056495127581,
'occupation_Other': 0.004026761173288829,
'occupation_Retail Worker': 2.854091147775997,
'occupation_Software Developer': 0.8015880293811709,
'occupation_Teacher': 1.1108173028665436}
```

```
In [22]: total_sum = sum(allocated_scores.values())  
total_sum
```

Out[22]: 1000.0

```
In [23]: total_location_score = sum(value for key, value in allocated_scores.items())  
print(f"Total location value: {total_location_score}")
```

Total location value: 16.938472509208665

```
In [24]: total_occupation_score = sum(value for key, value in allocated_scores.items())  
print(f"Total occupation value: {total_occupation_score}")
```

Total occupation value: 14.359797215796819

```
In [25]: max_location_score = max(value for key, value in allocated_scores.items())  
print(f"Maximum location value: {max_location_score}")
```

Maximum location value: 3.5813740336948414

```
In [26]: max_occupation_score = max(value for key, value in allocated_scores.items())  
print(f"Maximum occupation value: {max_occupation_score}")
```

Maximum occupation value: 3.282056495127581


```

In [27]: def compute_score(row, allocated_scores, df):
    total_score = 0

    numeric_columns_plus = [
        'debt_to_income_ratio', 'number_of_delinquent_accounts',
        'number_of_defaulted_accounts', 'new_accounts_opened_last_12months',
        'delinquency_status_log', 'number_of_credit_applications_log',
        'recent_trade_activity_month', 'earliest_credit_account_month',
        'account_open_date_year'
    ]

    numeric_columns_minus = [
        'age', 'income_level',
        'fico_score', 'max_balance_log',
        'avg_balance_last_12months_capped', 'recent_trade_activity_year',
        'recent_trade_activity_day_of_week', 'earliest_credit_account_year',
        'earliest_credit_account_day_of_week', 'account_open_date_month',
        'account_open_date_day_of_week'
    ]

    for col in numeric_columns_plus:
        min_val = df[col].min()
        max_val = df[col].max()
        normalized_score = ((row[col] - min_val) / (max_val - min_val)) if r
        total_score += (normalized_score) * allocated_scores[col]

    for col in numeric_columns_minus:
        min_val = df[col].min()
        max_val = df[col].max()
        normalized_score = ((row[col] - min_val) / (max_val - min_val)) if r
        total_score += (1-normalized_score) * allocated_scores[col]

    # One-hot encoded categorical columns: Contribution to the score based o
    boolean_columns_plus = [
        'location_Atlanta', 'location_Miami',
        'location_New York', 'location_San Francisco',
        'location_Seattle', 'occupation_Accountant',
        'occupation_Artist', 'occupation_Banker',
        'occupation_Consultant', 'occupation_Doctor',
        'occupation_Engineer', 'occupation_Other',
        'watchlist_blacklist_flag', 'public_records_flag',
        'date_contradiction_flag', 'payment_methods_high_risk',
        'multiple_applications_short_time_period', 'unusual_submission_patte
        'applications_submitted_during_odd_hours'
    ]

    boolean_columns_minus = [
        'location_Boston', 'location_Chicago',
        'location_Dallas', 'location_Houston',
        'location_Los Angeles', 'occupation_Lawyer',
        'occupation_Retail Worker', 'occupation_Software Developer',
        'occupation_Teacher'
    ]

    boolean_columns_plus_score=0
    boolean_columns_minus_score=0

    for col in boolean_columns_plus:

```

```

categorical_score = row[col] # Value is 1 or 0 in one-hot encoding
if col.startswith('location'):
    boolean_columns_plus_score += (categorical_score) * allocated_s
elif col.startswith('occupation'):
    boolean_columns_plus_score += (categorical_score) * allocated_s
else :
    boolean_columns_plus_score += (categorical_score) * allocated_s

for col in boolean_columns_minus:
    categorical_score = row[col] # Value is 1 or 0 in one-hot encoding
    if col.startswith('location'):
        boolean_columns_minus_score += (categorical_score) * allocated_s
    elif col.startswith('occupation'):
        boolean_columns_minus_score += (categorical_score) * allocated_s

total_score += boolean_columns_plus_score - boolean_columns_minus_score
return total_score

# Apply the function to compute scores for all rows
df_train['Actual_score'] = df_train.apply(compute_score, axis=1, allocated_s

```

In [28]: `df_train['charge_off_status'] =df_train.pop('charge_off_status')`

In [29]: `df_train`

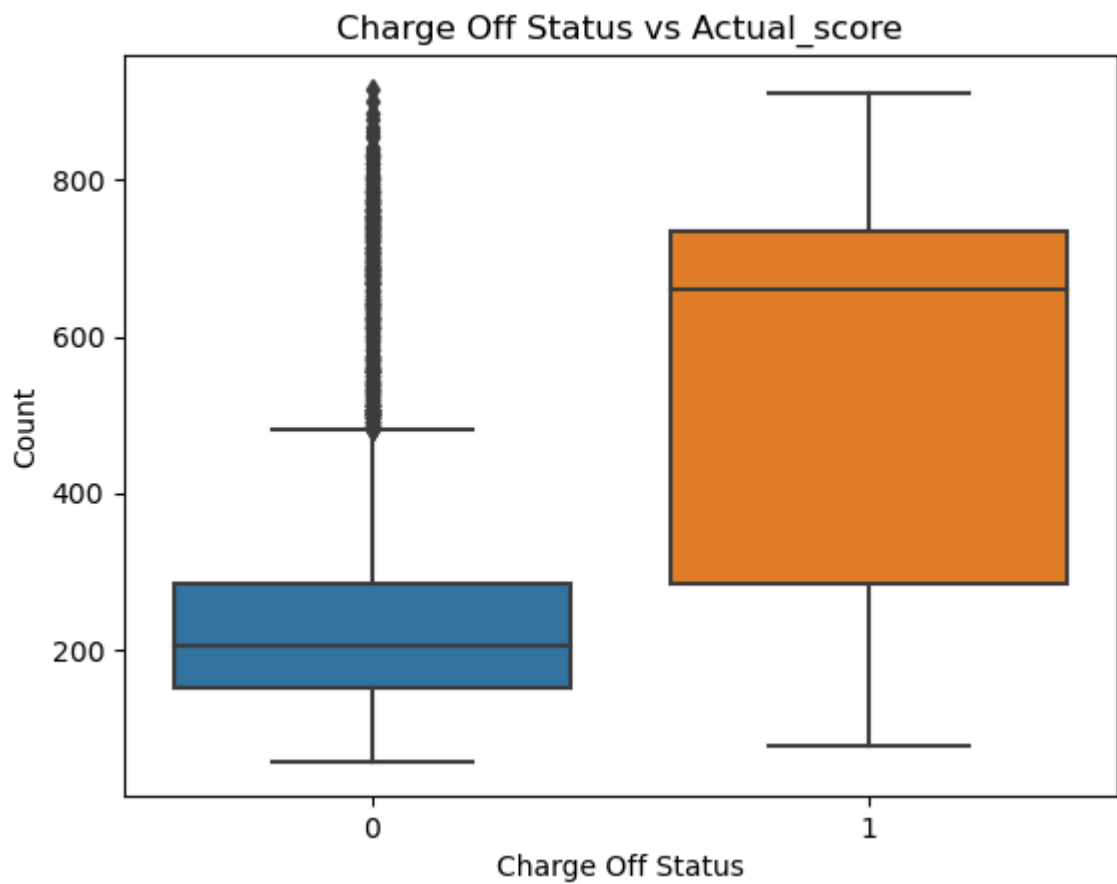
Out[29]:

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	numb
0	46	71936	566.0	1.508626		0
1	60	8574	787.0	1.182380		0
2	25	59349	688.0	1.121196		1
3	38	6925	740.0	1.659631		0
4	36	6197	758.0	1.008080		0
...	...	...	...	...		...
6715	65	10876	447.0	2.214227		1
6716	29	16347	469.0	1.516757		1
6717	74	51012	580.0	2.648878		1
6718	33	14566	589.0	1.727134		1
6719	68	4501	321.0	1.750392		1

6720 rows × 50 columns

In [30]:

```
sns.boxplot(x=df_train['charge_off_status'], y=df_train['Actual_score'])  
plt.title('Charge Off Status vs Actual_score')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```

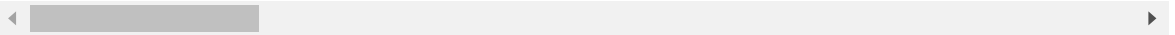


In [31]: df_train

Out[31]:

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	numt
0	46	71936	566.0	1.508626		0
1	60	8574	787.0	1.182380		0
2	25	59349	688.0	1.121196		1
3	38	6925	740.0	1.659631		0
4	36	6197	758.0	1.008080		0
...	...	...	...	...		...
6715	65	10876	447.0	2.214227		1
6716	29	16347	469.0	1.516757		1
6717	74	51012	580.0	2.648878		1
6718	33	14566	589.0	1.727134		1
6719	68	4501	321.0	1.750392		1

6720 rows × 50 columns



In [32]: df_charge_off_1 = df_train[df_train['charge_off_status'] == 1]
df_charge_off_0 = df_train[df_train['charge_off_status'] == 0]

In [33]: df_charge_off_1['Actual_score'].max()

Out[33]: 911.3925171952835

In [34]: df_charge_off_1['Actual_score'].quantile(0.25)

Out[34]: 284.2199820530268

In [35]: df_charge_off_0['Actual_score'].quantile(0.75)

Out[35]: 284.1075982370014

In [36]: df_charge_off_1['Actual_score'].min()

Out[36]: 76.51369802706564

In [37]: df_charge_off_0['Actual_score'].max()

Out[37]: 916.1328859520934

In [38]: df_charge_off_0['Actual_score'].min()

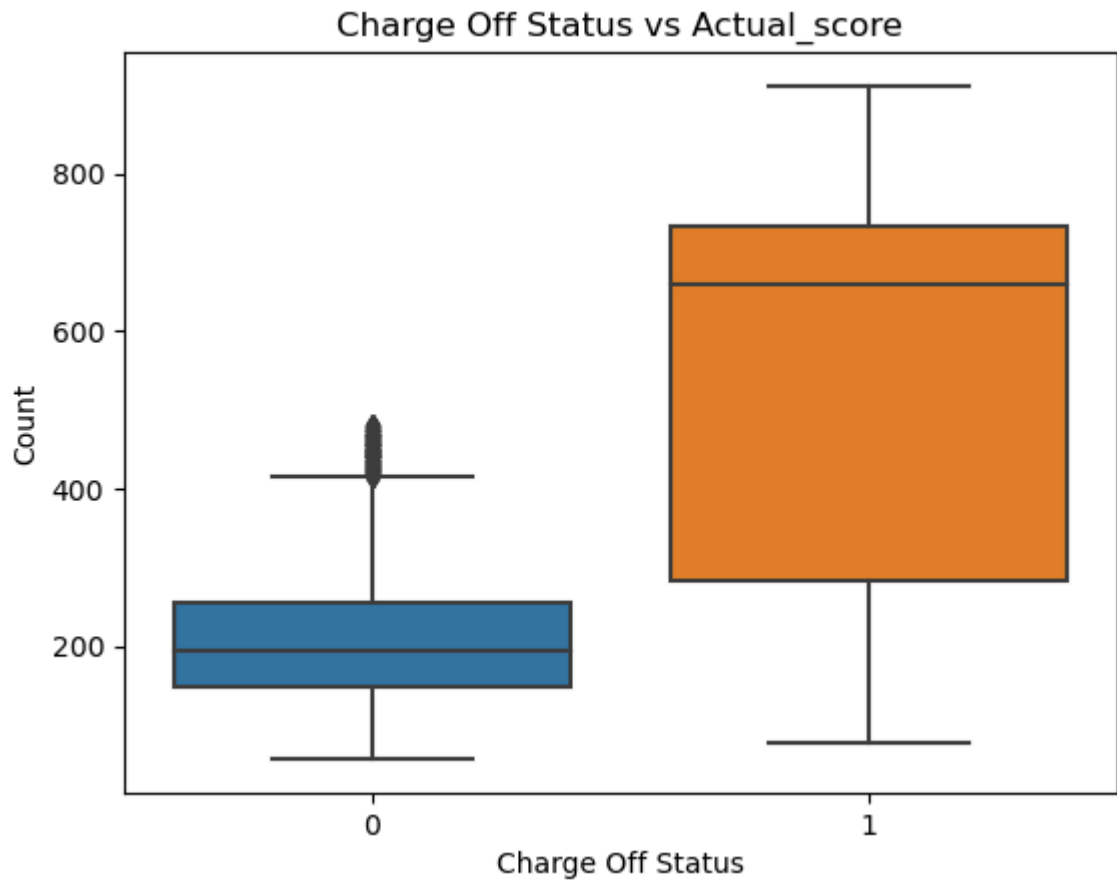
Out[38]: 57.00537604920625


```
In [39]: outliers_Actual_score_1 = df_charge_off_1[((df_charge_off_1['Actual_score']  
outliers_Actual_score_0 = df_charge_off_0[((df_charge_off_0['Actual_score']  
print(f"outliers_Actual_score_1 :{outliers_Actual_score_1.shape[0]}")  
print(f"outliers_Actual_score_0 :{outliers_Actual_score_0.shape[0]}")
```

```
outliers_Actual_score_1 :0  
outliers_Actual_score_0 :138
```

```
In [40]:  
  
upper_bound_value_0 = df_charge_off_0['Actual_score'].quantile(0.75) + 1.5  
upper_bound_value_1 = df_charge_off_1['Actual_score'].quantile(0.75) + 1.5  
lower_bound_value_0 = df_charge_off_0['Actual_score'].quantile(0.25) - 1.5  
lower_bound_value_1 = df_charge_off_1['Actual_score'].quantile(0.25) - 1.5  
  
df_charge_off_1 = df_charge_off_1[(df_charge_off_1['Actual_score'] >= lower  
df_charge_off_0 = df_charge_off_0[(df_charge_off_0['Actual_score'] >= lower  
# Apply capping  
#df_charge_off_0['Actual_score'] = np.where(df_charge_off_0['Actual_score']  
#df_charge_off_1['Actual_score'] = np.where(df_charge_off_1['Actual_score']  
  
# Concatenate the filtered dataframes  
df_train = pd.concat([df_charge_off_0, df_charge_off_1])  
  
df_train = df_train.reset_index(drop=True)
```

```
In [41]: sns.boxplot(x=df_train['charge_off_status'], y=df_train['Actual_score'])  
plt.title('Charge Off Status vs Actual_score')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```



```
In [42]: df_charge_off_1 = df_train[df_train['charge_off_status'] == 1]  
df_charge_off_0 = df_train[df_train['charge_off_status'] == 0]
```

```
In [43]: df_charge_off_1['Actual_score'].max()
```

```
Out[43]: 911.3925171952835
```

```
In [44]: df_charge_off_1['Actual_score'].quantile(0.25)
```

```
Out[44]: 284.2199820530268
```

```
In [45]: df_charge_off_0['Actual_score'].quantile(0.75)
```

```
Out[45]: 254.83618387753012
```

```
In [46]: df_charge_off_0['Actual_score'].max()
```

```
Out[46]: 481.9785963896943
```

```
In [47]: df_charge_off_0['Actual_score'].min()
```

```
Out[47]: 57.00537604920625
```

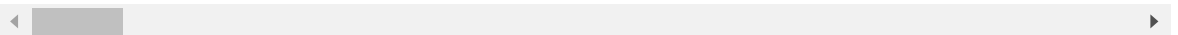
```
In [48]: pd.set_option('display.max_columns', 50)
```

```
In [49]: df_train
```

Out[49]:

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	numt
0	46	71936	566.0	1.508626		0
1	60	8574	787.0	1.182380		0
2	25	59349	688.0	1.121196		1
3	38	6925	740.0	1.659631		0
4	36	6197	758.0	1.008080		0
...	...	...	...	...		...
6251	65	10876	447.0	2.214227		1
6252	29	16347	469.0	1.516757		1
6253	74	51012	580.0	2.648878		1
6254	33	14566	589.0	1.727134		1
6255	68	4501	321.0	1.750392		1

6256 rows × 50 columns



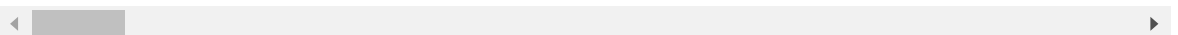
```
In [51]: # Drop columns 'x' for train model
X = df_train.drop(['Actual_score', 'charge_off_status'], axis=1)
```

```
In [52]: X
```

Out[52]:

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	numt
0	46	71936	566.0	1.508626		0
1	60	8574	787.0	1.182380		0
2	25	59349	688.0	1.121196		1
3	38	6925	740.0	1.659631		0
4	36	6197	758.0	1.008080		0
...	...	...	...	...		...
6251	65	10876	447.0	2.214227		1
6252	29	16347	469.0	1.516757		1
6253	74	51012	580.0	2.648878		1
6254	33	14566	589.0	1.727134		1
6255	68	4501	321.0	1.750392		1

6256 rows × 48 columns



In [53]:

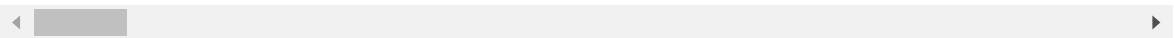
```
# Apply MinMaxScaler
scaler = MinMaxScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

X

Out[53]:

	age	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk
0	0.500000	0.715663	0.484461	0.375953	0.0
1	0.750000	0.067272	0.888483	0.261346	0.0
2	0.125000	0.586859	0.707495	0.239853	1.0
3	0.357143	0.050398	0.802559	0.428999	0.0
4	0.321429	0.042948	0.835466	0.200117	0.0
...	...	...	...	...	...
6251	0.839286	0.090829	0.266910	0.623821	1.0
6252	0.196429	0.146814	0.307130	0.378809	1.0
6253	1.000000	0.501545	0.510055	0.776509	1.0
6254	0.267857	0.128589	0.526508	0.452712	1.0
6255	0.892857	0.025593	0.036563	0.460882	1.0

6256 rows × 48 columns

In [54]: `y = df_train['Actual_score']`

```
In [77]: from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor, GradientBoostingRegressor
```


In [107]:

```
# Updated dictionary for regression tasks
algos = {
    'linear_regression': {
        'model': LinearRegression(),
        'params': {
            'fit_intercept': [True, False],
            'normalize': [True, False]
        }
    },
    'ridge_regression': {
        'model': Ridge(),
        'params': {
            'alpha': [0.1, 1, 10],
            'solver': ['auto', 'svd', 'cholesky', 'lsqr', 'saga']
        }
    },
    'lasso_regression': {
        'model': Lasso(),
        'params': {
            'alpha': [0.1, 1, 10],
            'max_iter': [1000, 5000]
        }
    },
    'SVM': {
        'model': SVR(),
        'params': {
            'C': [0.1, 1, 10],
            'kernel': ['linear', 'rbf', 'poly'],
            'epsilon': [0.1, 0.2, 0.5]
        }
    },
    'decision_tree': {
        'model': DecisionTreeRegressor(),
        'params': {
            'criterion': ['squared_error', 'friedman_mse', 'absolute_error'],
            'splitter': ['best', 'random']
        }
    },
    'random_forest': {
        'model': RandomForestRegressor(),
        'params': {
            'n_estimators': [10, 50, 100],
            'criterion': ['squared_error', 'absolute_error', 'poisson']
        }
    },
    'ada_boost_regressor': {
        'model': AdaBoostRegressor(),
        'params': {
            'n_estimators': [50, 100, 200],
            'learning_rate': [0.01, 0.1, 1]
        }
    },
    'gradient_boosting_regressor': {
        'model': GradientBoostingRegressor(),
        'params': {
            'n_estimators': [50, 100, 200],
            'loss': ['squared_error', 'absolute_error', 'huber'],
            'learning_rate': [0.01, 0.1, 0.2]
        }
    }
}
```

```

    },
    'bagging_regressor': {
        'model': BaggingRegressor(),
        'params': {
            'n_estimators': [10, 50, 100],
            'max_samples': [0.5, 1.0],
            'max_features': [0.5, 1.0]
        }
    }
}

```

In [108]:

```

In [109]: scores= []
cv = ShuffleSplit(n_splits=5, test_size=0.2, random_state=10)
for algo_name, mp in algos.items():
    reg = GridSearchCV(mp['model'], mp['params'], cv=cv, return_train_score=False)
    reg.fit(X, y)
    scores.append(
        {
            'model': algo_name,
            'best_score': reg.best_score_,
            'best_params': reg.best_params_
        }
    )

```

If you wish to scale the data, use Pipeline with a StandardScaler in a preprocessing stage. To reproduce the previous behavior:

```
from sklearn.pipeline import make_pipeline
```

```
model = make_pipeline(StandardScaler(with_mean=False), LinearRegression())
```

If you wish to pass a sample_weight parameter, you need to pass it as a fit parameter to each step of the pipeline as follows:

```
kwargs = {s[0] + '__sample_weight': sample_weight for s in model.steps}
model.fit(X, y, **kwargs)
```

```
warnings.warn(
C:\Users\THUSHAN\anaconda3\lib\site-packages\sklearn\linear_model\_base.
py:141: FutureWarning: 'normalize' was deprecated in version 1.0 and will
be removed in 1.2.
```

If you wish to scale the data, use Pipeline with a StandardScaler in a preprocessing stage.

```
In [110]: score = pd.DataFrame(scores, columns=['model', 'best_score', 'best_params'])
score
```

```
Out[110]:
```

	model	best_score	best_params
0	linear_regression	1.000000	{'fit_intercept': True, 'normalize': True}
1	ridge_regression	1.000000	{'alpha': 0.1, 'solver': 'auto'}
2	lasso_regression	0.999920	{'alpha': 0.1, 'max_iter': 1000}
3	SVM	1.000000	{'C': 1, 'epsilon': 0.1, 'kernel': 'linear'}
4	decision_tree	0.977773	{'criterion': 'squared_error', 'splitter': 'ra...
5	random_forest	0.989149	{'criterion': 'squared_error', 'n_estimators':...
6	ada_boost_regressor	0.962312	{'learning_rate': 1, 'n_estimators': 100}
7	gradient_boosting_regressor	0.998394	{'learning_rate': 0.2, 'loss': 'squared_error'...
8	bagging_regressor	0.989235	{'max_features': 1.0, 'max_samples': 1.0, 'n_e...

```
In [55]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
```

```
In [79]:
```

```
model = LinearRegression(fit_intercept=True, normalize=True)
model.fit(X_train, y_train)
```

C:\Users\THUSHAN\anaconda3\lib\site-packages\sklearn\linear_model_base.p
y:141: FutureWarning: 'normalize' was deprecated in version 1.0 and will b
e removed in 1.2.

If you wish to scale the data, use Pipeline with a StandardScaler in a pre
processing stage. To reproduce the previous behavior:

```
from sklearn.pipeline import make_pipeline
```

```
model = make_pipeline(StandardScaler(with_mean=False), LinearRegression())
```

If you wish to pass a sample_weight parameter, you need to pass it as a fi
t parameter to each step of the pipeline as follows:

```
kwargs = {s[0] + '__sample_weight': sample_weight for s in model.steps}
model.fit(X, y, **kwargs)
```

```
warnings.warn(
```

```
Out[79]: LinearRegression(normalize=True)
```

```
In [80]: model.score(X_test, y_test)
```

```
Out[80]: 1.0
```

```
In [82]: y_pred = model.predict(X_test)
```


In [83]:

```
# Assuming 'y_pred' are your predicted values and 'y_test' are the true values
compare = pd.DataFrame({
    'Actual': y_test,
    'Predicted': y_pred
})

compare
```

Out[83]:

	Actual	Predicted
2284	176.753070	176.753070
4814	195.864347	195.864347
3946	264.827156	264.827156
1608	218.284978	218.284978
5487	446.199261	446.199261
...	...	...
4836	88.192459	88.192459
5798	678.903289	678.903289
1024	125.298302	125.298302
3661	127.655720	127.655720
4205	242.773794	242.773794

1252 rows × 2 columns

In [84]:

```
# Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# R-squared (R^2)
r_squared = r2_score(y_test, y_pred)

# Print the evaluation metrics
print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"R-squared (R^2): {r_squared}")
```

```
Mean Absolute Error (MAE): 1.336864591339051e-13
Mean Squared Error (MSE): 2.934252063704837e-26
Root Mean Squared Error (RMSE): 1.7129658676415116e-13
R-squared (R^2): 1.0
```

In [85]: y_test

```
Out[85]: 2284    176.753070
         4814    195.864347
         3946    264.827156
         1608    218.284978
         5487    446.199261
         ...
         4836     88.192459
         5798    678.903289
         1024    125.298302
         3661    127.655720
         4205    242.773794
Name: Actual_score, Length: 1252, dtype: float64
```

In [86]:

```
fig, ax = plt.subplots(figsize=(10, 6))

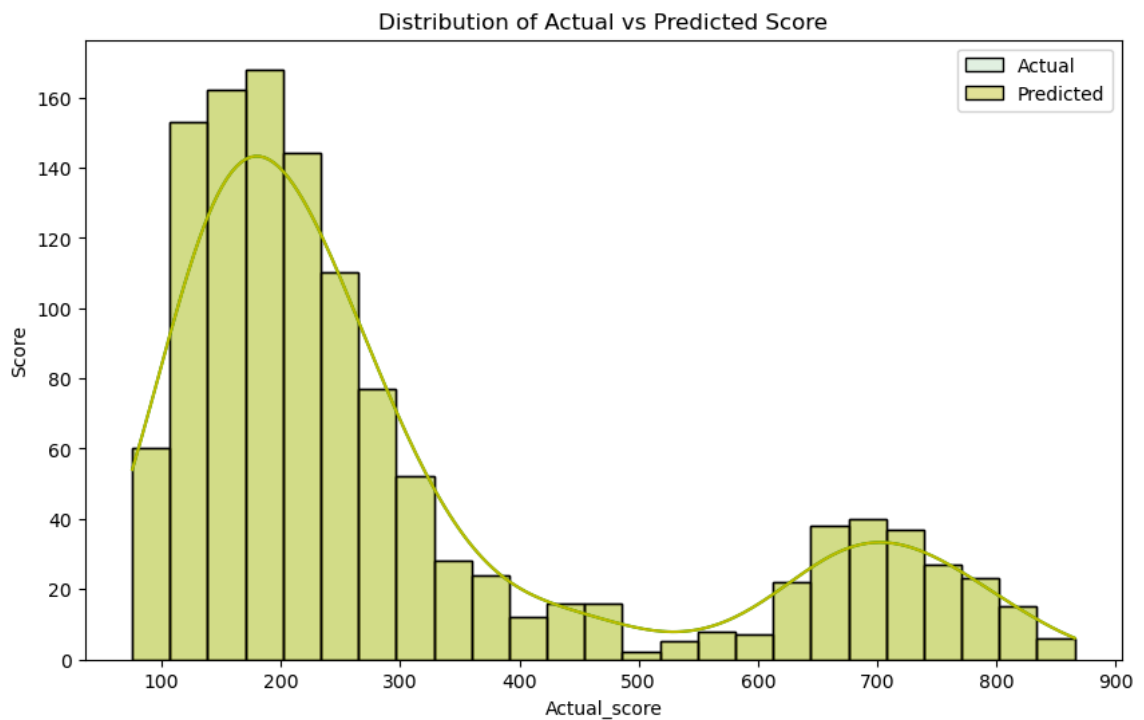
# Plot the distribution of the 'Actual'
sns.histplot(y_test, color='g', label='Actual', alpha=0.1, kde=True, ax=ax)

# Plot the distribution of the 'Predicted'
sns.histplot(y_pred, color='y', label='Predicted', alpha=0.4, kde=True, ax=ax)

ax.legend()

ax.set_title('Distribution of Actual vs Predicted Score')
ax.set_ylabel('Score')

plt.show()
```



```
In [65]: pca = PCA()  
# Train model  
pca.fit(X)  
# New data PCA_Scales  
x_pca = pca.transform(X)
```

```
In [66]: x_pca = pd.DataFrame(x_pca)
```

```
In [67]: # Now, analyze the variance of data_scaler_pca  
var = pca.explained_variance_ratio_  
# Show the variance of all features  
print(len(var))  
print(var)
```

48

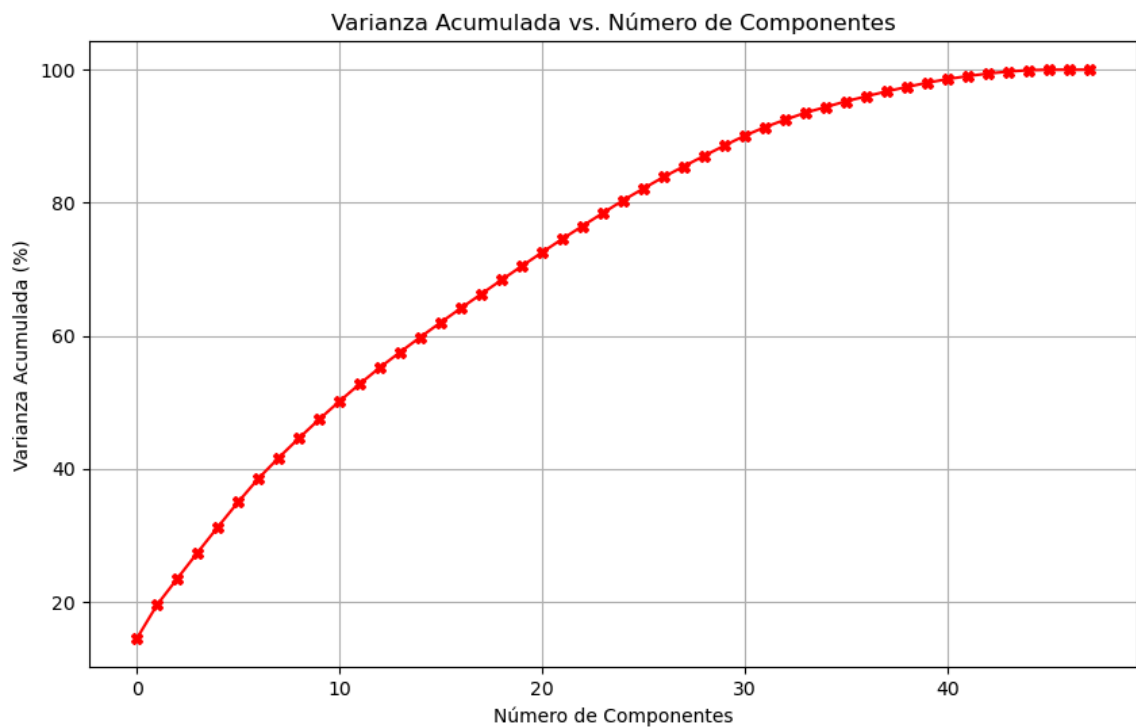
```
[1.45002178e-01 5.05887779e-02 3.95810719e-02 3.90645537e-02  
3.83479468e-02 3.74039867e-02 3.54299363e-02 3.11917282e-02  
2.94305621e-02 2.82771481e-02 2.70930612e-02 2.61267184e-02  
2.48493478e-02 2.30709344e-02 2.26087745e-02 2.19305291e-02  
2.15163487e-02 2.12427540e-02 2.11458407e-02 2.08157393e-02  
2.05595909e-02 2.02490750e-02 1.94951454e-02 1.94361917e-02  
1.91175535e-02 1.77001971e-02 1.74480343e-02 1.60039085e-02  
1.58221763e-02 1.52655609e-02 1.48674956e-02 1.28487678e-02  
1.12174491e-02 1.05331159e-02 8.64679207e-03 8.51949103e-03  
7.64018590e-03 7.28406531e-03 6.74557427e-03 6.26327602e-03  
5.49095886e-03 4.54598318e-03 3.87306598e-03 2.90198264e-03  
2.07871608e-03 7.27708573e-04 2.85966374e-32 6.11363335e-33]
```

```
In [69]: # Chaeck the best option to reduce the dimensionality, for this we plot the
cum_var = np.cumsum(np.round(var, decimals = 4) * 100) # np.cumsum is the cu
# Show results
for i, acumulado in enumerate(cum_var, start=1):
    print(f"{i} componente: {acumulado:.2f}%")
plt.figure(figsize=(10, 6))
plt.plot(cum_var, 'r-x', marker='X')
plt.xlabel('Número de Componentes')
plt.ylabel('Varianza Acumulada (%)')
plt.title('Varianza Acumulada vs. Número de Componentes')
plt.grid(True)
plt.show()
```

1 componente: 14.50%
2 componente: 19.56%
3 componente: 23.52%
4 componente: 27.43%
5 componente: 31.26%
6 componente: 35.00%
7 componente: 38.54%
8 componente: 41.66%
9 componente: 44.60%
10 componente: 47.43%
11 componente: 50.14%
12 componente: 52.75%
13 componente: 55.23%
14 componente: 57.54%
15 componente: 59.80%
16 componente: 61.99%
17 componente: 64.14%
18 componente: 66.26%
19 componente: 68.37%
20 componente: 70.45%
21 componente: 72.51%
22 componente: 74.53%
23 componente: 76.48%
24 componente: 78.42%
25 componente: 80.33%
26 componente: 82.10%
27 componente: 83.84%
28 componente: 85.44%
29 componente: 87.02%
30 componente: 88.55%
31 componente: 90.04%
32 componente: 91.32%
33 componente: 92.44%
34 componente: 93.49%
35 componente: 94.35%
36 componente: 95.20%
37 componente: 95.96%
38 componente: 96.69%
39 componente: 97.36%
40 componente: 97.99%
41 componente: 98.54%
42 componente: 98.99%
43 componente: 99.38%
44 componente: 99.67%
45 componente: 99.88%
46 componente: 99.95%
47 componente: 99.95%
48 componente: 99.95%

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2712\760473514.py:7: UserWarning: marker is redundantly defined by the 'marker' keyword argument and the fmt string "r-x" (-> marker='x'). The keyword argument will take precedence.

```
plt.plot(cum_var, 'r-x', marker='X')
```



In [70]: `x_pca.head()`

Out[70]:

	0	1	2	3	4	5	6	7	
0	-0.416284	0.610907	-0.131448	0.180366	0.088729	-0.267377	0.038917	0.392772	0.57
1	-0.278293	-0.202318	-0.141199	0.074672	-0.568406	0.043829	-0.498666	0.272953	-0.65
2	-0.308057	-0.327081	-0.557362	-0.039077	-0.417038	-0.340205	-0.841242	0.798338	0.12
3	-0.484726	-0.188280	0.642744	-0.512095	-0.623386	0.156028	-0.746670	-0.073726	-0.33
4	0.107968	-0.198134	0.423468	0.889395	0.403739	0.787364	-0.681869	-0.016329	-0.30

In [71]: `# Drop components provide no information`
`x_pca.drop([40,41, 42, 43,44,45,46,47], axis = 1, inplace = True)`

In [72]: `x_pca.head()`

Out[72]:

	0	1	2	3	4	5	6	7	
0	-0.416284	0.610907	-0.131448	0.180366	0.088729	-0.267377	0.038917	0.392772	0.57
1	-0.278293	-0.202318	-0.141199	0.074672	-0.568406	0.043829	-0.498666	0.272953	-0.65
2	-0.308057	-0.327081	-0.557362	-0.039077	-0.417038	-0.340205	-0.841242	0.798338	0.12
3	-0.484726	-0.188280	0.642744	-0.512095	-0.623386	0.156028	-0.746670	-0.073726	-0.33
4	0.107968	-0.198134	0.423468	0.889395	0.403739	0.787364	-0.681869	-0.016329	-0.30

```
In [73]: X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(x_pca, y
```

```
In [74]: model = LinearRegression(fit_intercept=True, normalize=True)  
model.fit(X_train_pca, y_train_pca)
```

C:\Users\THUSHAN\anaconda3\lib\site-packages\sklearn\linear_model_base.p
y:141: FutureWarning: 'normalize' was deprecated in version 1.0 and will b
e removed in 1.2.

If you wish to scale the data, use Pipeline with a StandardScaler in a pre
processing stage. To reproduce the previous behavior:

```
from sklearn.pipeline import make_pipeline
```

```
model = make_pipeline(StandardScaler(with_mean=False), LinearRegression())
```

If you wish to pass a sample_weight parameter, you need to pass it as a fi
t parameter to each step of the pipeline as follows:

```
kwargs = {s[0] + '__sample_weight': sample_weight for s in model.steps}  
model.fit(X, y, **kwargs)
```

```
warnings.warn(
```

```
Out[74]: LinearRegression(normalize=True)
```

```
In [75]: model.score(X_test_pca, y_test_pca)
```

```
Out[75]: 0.9987274739703366
```

```
In [ ]:
```